حصن التطبيقات من الثغرات والآفات

Lecture 1 — Application Security

> **Application security** : the set of measures and practices that protect software applications from threats and vulnerabilities **throughout their life cycle**. It is a specific subset of cybersecurity focused on securing **software** itself.

Area	What it protects	Typical measures
Network security	Traffic and connections between systems	Firewalls, segmentation, IDS, VPNs
Infrastructure / cloud	Servers, containers, VMs, cloud accounts	Hardening, patching, IAM, configuration baselines
Physical security	Buildings, data centers, devices	Badges, locks, cameras
Application security	The software: design, code, dependencies, runtime behaviour	Threat modeling, secure coding, code review, SAST/DAST, dependency scanning, monitoring

> **Shift-left** : moving security activities **earlier** in the life cycle so feedback arrives while the code is still fresh.

Security objectives

> **Confidentiality** : data is accessible only to authorized parties.

> **Integrity** : data cannot be modified by unauthorized parties.

> **Availability** : the application and its data are accessible when legitimate users need them.

> **Authentication** : verifying a user, device or system **is who it claims** to be.

> **Authorization** : deciding what an **already-authenticated** identity may do or see.

> **Accountability** : trustworthy evidence of **who did what and when**, so actions can be reconstructed and not denied.

Objective	Core question	Mechanisms	Typical failures
Confidentiality	Can an unauthorized party read it?	Encryption, access control, data classification	Missing access control, unencrypted data, verbose errors, IDOR
Integrity	Can it be changed without authorization?	Input validation, digital signatures, audit trails	No input validation, no signatures, SQL injection
Availability	Can users reach it when needed?	Rate limiting, redundancy, capacity planning	DoS, resource exhaustion, unpatched crashes
Authentication	Who are you?	Passwords, MFA, certificates	Weak/reused passwords, no MFA, predictable tokens
Authorization	What may you do?	RBAC, policy engines	Missing object-level checks, hidden buttons instead of server checks, broad roles
Accountability	Can we prove who did what, when?	Audit logs, timestamps, correlation IDs	Missing logs, no user/time, tamperable logs

Example: a support agent who changes an account ID in the URL and sees every company's invoices — authenticated correctly, **authorized** wrongly.

Q/ What are the three pillars of application security administration ?

Pillar	Role	Examples
People	Responsibility and decisions	Developers, security engineers, architects, Security Advocate, app owners
Process	Consistent security activities	Threat modeling, code review, vulnerability & patch management, incident response
Technology	Automated detection	SAST, DAST, SCA, secrets scanning, runtime monitoring, CI/CD controls

Model	Core idea	Representative controls
Zero Trust	No implicit trust; access continuously evaluated	Strong identity, service-to-service auth, signed tokens, just-in-time privileges
Shift-Left	Security earlier in the life cycle	SAST in IDE/PR, dependency analysis, secrets scanning, pre-merge tests
Defense in Depth	Many independent layers; no single failure = full compromise	Validation + parameterized queries + least-privilege DB + authz + monitoring
DevSecOps	Security inside existing dev/ops workflows	Findings in the same tracker and CI/CD pipeline
Risk-Based	Prioritize by business context, not only technical severity	Asset criticality, data sensitivity, exploitability, exposure

Framework	Publisher	Focus
OWASP Top 10 / ASVS	OWASP	Common risks catalog + verification standard
NIST SSDF (SP 800-218)	NIST	Prepare, Protect, Produce, Respond
ISO/IEC 27034	ISO	App security inside the organization's ISMS
CIS Controls	Center for Internet Security	Prioritized prescriptive safeguards
PCI DSS	PCI SSC	Mandatory controls for payment card data

Threat modeling

> **Threat modeling** : a systematic process to **identify, understand, prioritize and mitigate** threats **before and throughout** implementation — proactive instead of reactive.

> **Vulnerability** : a weakness. **Threat** : a potential action/event that could exploit it. **Risk** : the potential consequences in context.

Q/ What are the three fundamental threat-modeling questions ?

1. **What are we building?** Components, users, assets, data flows, dependencies, trust boundaries.
2. **What can go wrong?** Threats to C, I, A, authentication, authorization, accountability.
3. **What are we going to do about it?** Preventive, detective, corrective controls; manage residual risk.

> **Data Flow Diagram (DFD)** : a structured picture of how information moves through the system, used to expose attack-relevant parts.

Q/ What are the elements of a security DFD ?

1. **External entities** (users, external apps, third parties).
2. **Processes** (components that receive/transform/validate data).
3. **Data stores** (databases, files, caches, queues, backups).
4. **Data flows** (exchanges between them).
5. **Trust boundaries** (the fifth, most important concept).

> **Trust boundary** : a logical line where the level of **trust, privilege or control changes** (not necessarily a network boundary).

Mobile app example: the device is **untrusted** — the server must independently authenticate, authorize and validate; never rely only on client-side checks.

> **STRIDE** : Microsoft's threat-modeling framework of six categories.

STRIDE	Definition	Property violated	Controls
Spoofing	Pretending to be another user/device/service	Authentication	MFA, credential & session management, token validation
Tampering	Unauthorized modification of data, requests, config or software	Integrity	Encrypted channels, server-side validation, signatures, DB constraints
Repudiation	Denying an action with no trustworthy evidence	Accountability / non-repudiation	Secure audit logs, timestamps, transaction IDs, tamper-resistant logs

STRIDE	Definition	Property violated	Controls
Information disclosure	Data reaches unauthorized parties	Confidentiality	Encryption, access control, least privilege, data minimization, safe errors
Denial of service	Users can't access the service / it degrades	Availability	Rate limiting, quotas, size limits, load balancing, DDoS protection
Elevation of privilege	Gaining permissions beyond those assigned	Authorization	Server-side authz, RBAC/ABAC, least privilege, object-level checks

> **Risk score** : **Likelihood × Impact**, each rated **High 3 · Medium 2 · Low 1** → score **1–9**.

Level	Likelihood	Impact
High (3)	Little skill/access; tools readily available	Many users, big loss, sensitive data, safety
Medium (2)	Some skill, privileged access or conditions	One user or limited group, moderate damage
Low (1)	Advanced skill, rare access, many conditions	Minor, temporary, reversible

Don't confuse the dimensions: a whole-database breach has high **impact** but isn't automatically **likely**. Don't rate everything High; justify each rating.

Q/ QuickBite worked example — what are the trust boundaries and top threats ?

- **TB1** Internet → QuickBite cloud (all client devices untrusted). **TB2** QuickBite → payment provider (authenticate both ways). **TB3** app tier → data tier.
- Top five: **I2** IDOR on /orders/5521→5522 (9), **S1** account takeover by credential stuffing (9), **T1** price tampering (9), **E1** customer calls restaurant endpoints (6), **D2** API/promo flooding at peak (6).
- Tie-break order: **personal safety → users affected → cost**.

Secure coding practices

Practice	Rule
Input validation	Never trust input; check type, length, format, range; prefer **allow-lists**
Parameterized queries	Keep SQL code separate from data (stops `' OR '1'='1`)
Output encoding	Encode special characters so the browser shows text, not code (XSS)
Password storage	Salted, slow hash (**bcrypt, Argon2**); never MD5/SHA-1
Authentication & sessions	MFA, lockout/slow-down, random session IDs, HttpOnly/Secure/SameSite, idle expiry
Authorization & least privilege	Check permission on **every request**; minimum rights (DB account can't DROP TABLE)
Secrets management	No hard-coded keys; load from env or a secrets manager; never commit to Git
Error handling & logging	Generic messages to users, details in protected logs, no sensitive data in logs
Encryption	HTTPS/TLS in transit, encryption at rest; never invent your own crypto
Secure defaults / fail-safe	Secure out of the box; on failure **fail closed** (deny)
Code review & testing	Peer review, SAST (Bandit, SonarQube), DAST, penetration testing
CSRF protection	Stop a logged-in browser being tricked into unwanted requests (tokens, SameSite)

Fail-safe example: if `check_permission` throws, set `allowed = False`, never `True`.

Summary of Advanced Cybersecurity L1: Application Security. Built only from the lecture file; check the original slides for anything you're unsure about.